크로스 플랫폼 프로그래밍



7주: HOOK

강의: 정종수



강의 내용

1. Hook
2. useState
3. useRef
4. useEffect
5. useMemo
6. 실습

1. Hook

React의 함수형 컴포넌트에서 상태(state)나 생명주기(lifecycle) 기능을 사용할 수 있게 해주는 특별한 함수.

- 함수 컴포넌트와 커스텀 Hook에서만 사용 가능
- 함수 최상위에서만 사용 가능
- Hook은 항상 use로 시작한다는 규칙

▪ 대표적인 Hook 종류

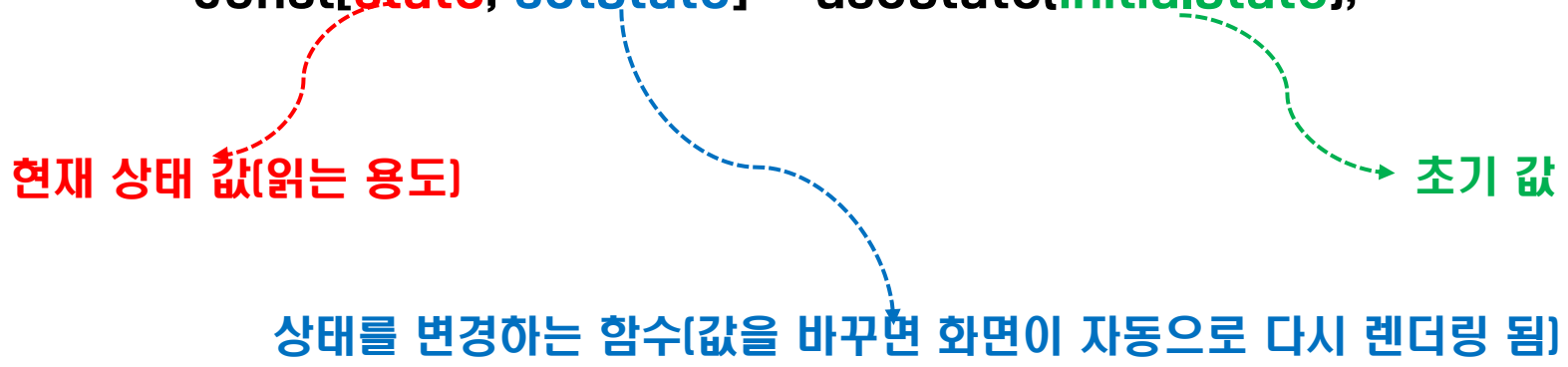
• useState	상태(state)관리	값 저장 및 업데이트
• useRef	참조관리	DOM 요소나 변수 값을 직접 제어
• useEffect	생명주기 제어	컴포넌트가 실행, 변경, 종료 때 코드 실행
• useMemo	참조관리	불필요한 재 계산 방지(성능 최적화)

2. useState

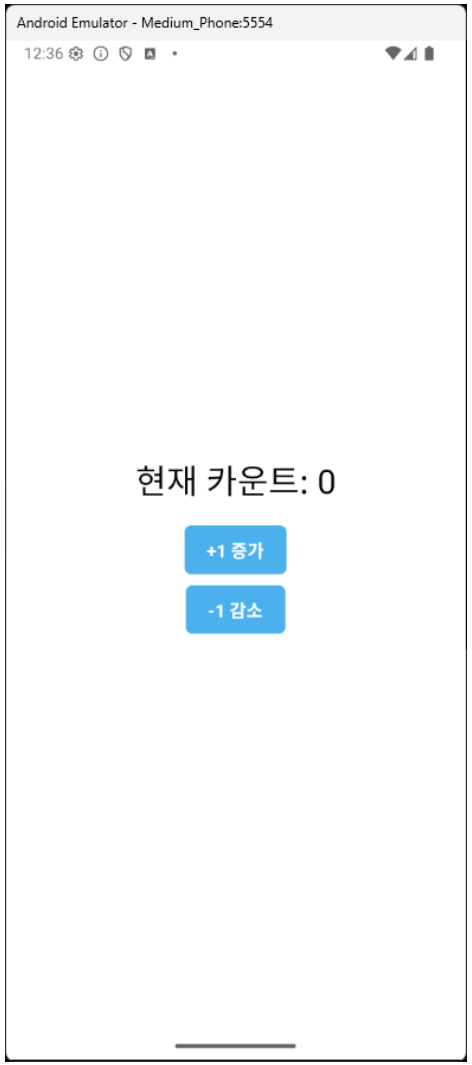
- useState 상태(state)관리 값 저장 및 업데이트

  - 예제 코드

```
const[state, setState] = useState(initialState);
```



2. useState: 예제 1.




```
import React, { useState } from 'react';
import { StyleSheet, Text, View, Pressable } from 'react-
native';

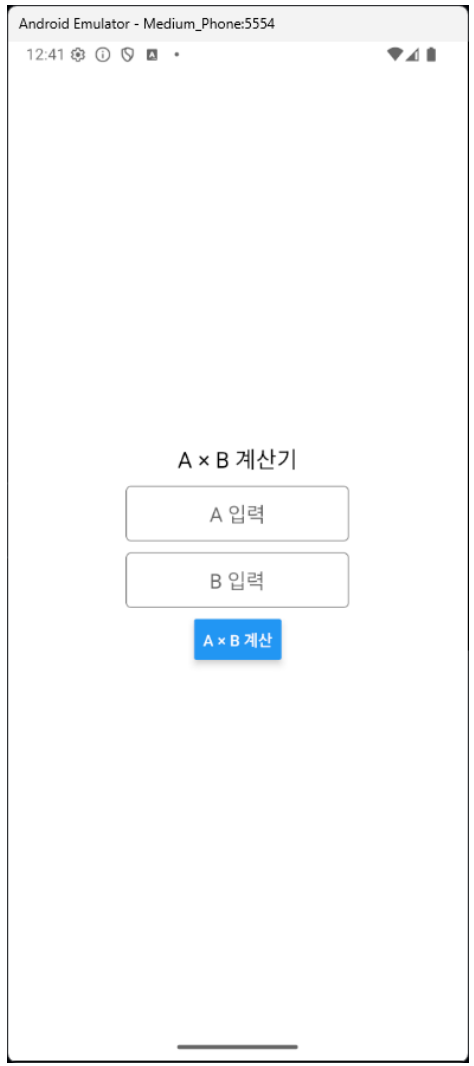

export default function App() {
  const [count, setCount] = useState(0);


  return (
    <View style={styles.container}>
      {/* 중앙 텍스트 */}
      <View style={{alignItems:'center'}}>
        <Text style={styles.text}>현재 카운트: {count}</Text>
      </View>


      <Pressable style={styles.button} onPress={() =>
setCount(count + 1)}>
        <Text style={styles.buttonText}>+1 증가</Text>
      </Pressable>
      <Pressable style={styles.button} onPress={() =>
setCount(count - 1)}>
        <Text style={styles.buttonText}>-1 감소</Text>
      </Pressable>
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent:'center',
    alignItems: 'center',
    padding: 30,
  },
  text: {
    fontSize: 30,
    marginBottom: 20,
  },
  button: {
    backgroundColor: '#4ab1ecff’,
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 6,
    alignItems: 'center',
    marginBottom:10,
  },
  buttonText: {
    color: '#fff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

2. useState: 예제 2.




```
import React, { useState } from 'react';
import { View, Text, TextInput, Button } from 'react-native';

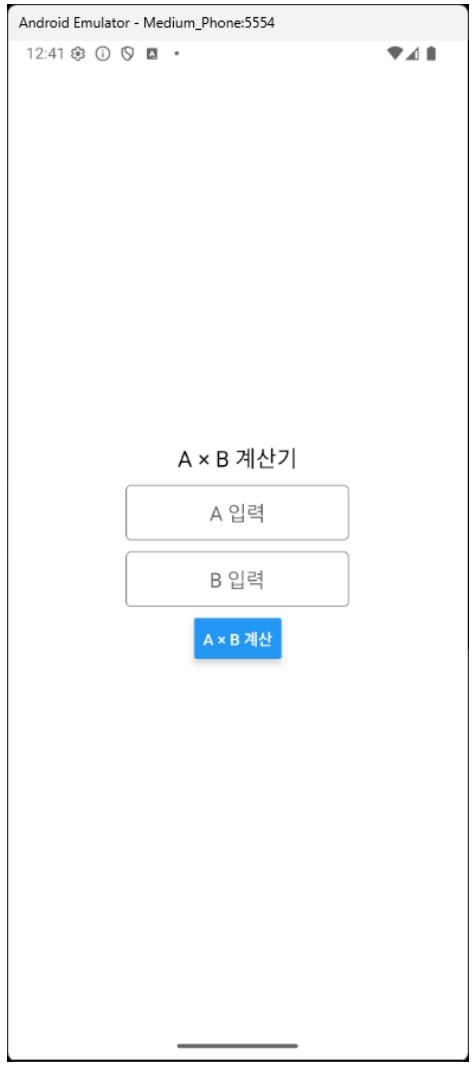

export default function App() {
  const [a, setA] = useState('');     // 첫 번째 숫자 (문자열)
  const [b, setB] = useState('');     // 두 번째 숫자 (문자열)
  const [result, setResult] = useState(null); // 계산 결과


  const multiply = () => {
    const numA = parseFloat(a);
    const numB = parseFloat(b);


    if (isNaN(numA) || isNaN(numB)) {
      setResult('숫자를 입력하세요!');
    } else {
      setResult(numA * numB);
    }
  };


  return (
    <View style={{ flex: 1, justifyContent: 'center',
alignItems: 'center' }}>
      <Text style={{ fontSize: 20, marginBottom: 10 }}>A × B
계산기</Text>
```

2. useState: 예제 2.




```
<TextInput
  style={{
    borderWidth: 1,
    borderColor: 'gray',
    width: 200,
    marginBottom: 10,
    textAlign: 'center',
    fontSize: 18,
    borderRadius: 5,
  }}
  keyboardType="numeric"
  placeholder="A 입력"
  value={a}
  onChangeText={setA}
/>

<TextInput
  style={{
    borderWidth: 1,
    borderColor: 'gray',
    width: 200,
    marginBottom: 10,
    textAlign: 'center',
    fontSize: 18,
    borderRadius: 5,
  }}
  keyboardType="numeric"
  placeholder="B 입력"
  value={b}
  onChangeText={setB}
/>
```

```
      <Button title="A × B 계산" onPress={multiply} />

      {result !== null && (
        <Text style={{ fontSize: 22, marginTop: 20 }}>결과:
        {result}</Text>
      )}
    </View>
  );
}
```

3. useRef

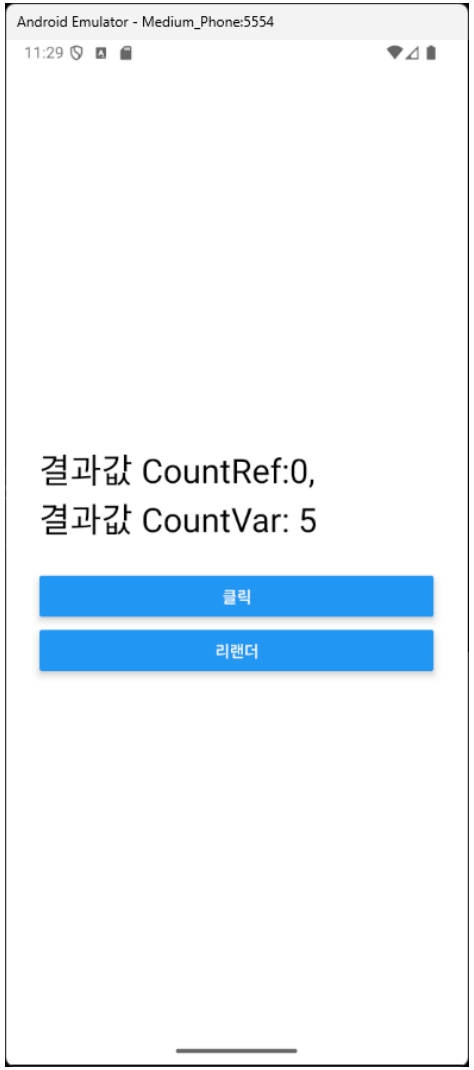
- useRef 참조관리 DOM 요소나 변수 값을 직접 제어

  - 예제 코드

    ```
    const ref = useRef(initiaValue);
    ```



3. useRef: 예제 1

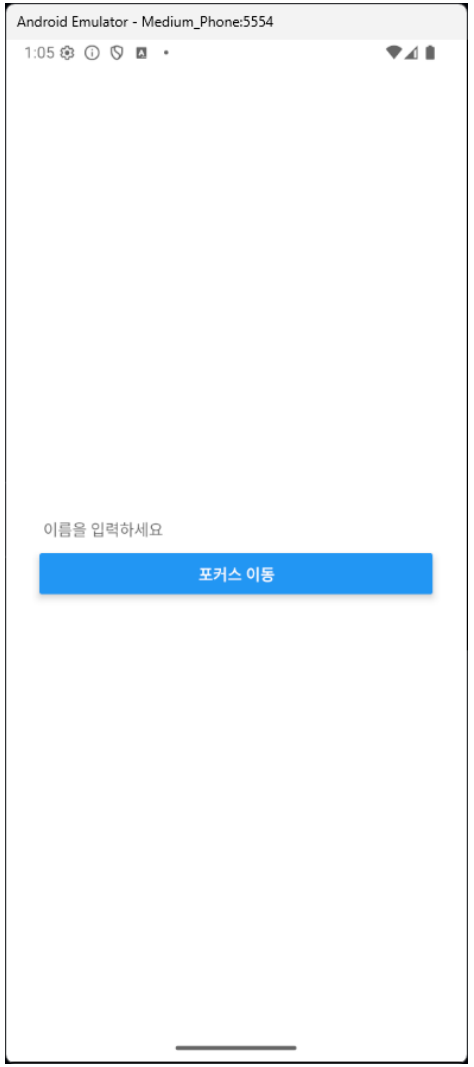



```
import React, { useRef, useState } from 'react';
import { View, Button, Text } from 'react-native';

export default function Example() {
  const [render, setRender] = useState(false);
  const countRef = useRef(0);
  let countVar = 5;

  const handleClick = () => {
    countRef.current += 1;
    countVar += 1;
    console.log('현재 count:', countRef.current, ": ", countVar);
  };
  const doRender = () =>{
    setRender(!render);
  }
  return (
    <View
      style={{
        flex: 1,                    // 전체 화면 차지
        justifyContent: 'center',   // 세로 중앙 정렬
        alignItems: 'stretch',      // 가로 중앙 정렬
        padding:30,
      }}>
      <Text style={{fontSize:30, marginTop:10, marginBottom:30}}>결과값 CountRef:{countRef.current}, {"₩n"}결과값 CountVar: {countVar}
</Text>
      <View style={{ marginBottom: 12 }}>
        <Button title="클릭" onPress={handleClick} />
      </View>
      <View>
        <Button title="리랜더" onPress={doRender} />
      </View>
    </View>
  );
}
```

3. useRef: 예제 2




```
import React, { useRef } from "react";
import {StyleSheet, View, TextInput, Button } from "react-native";

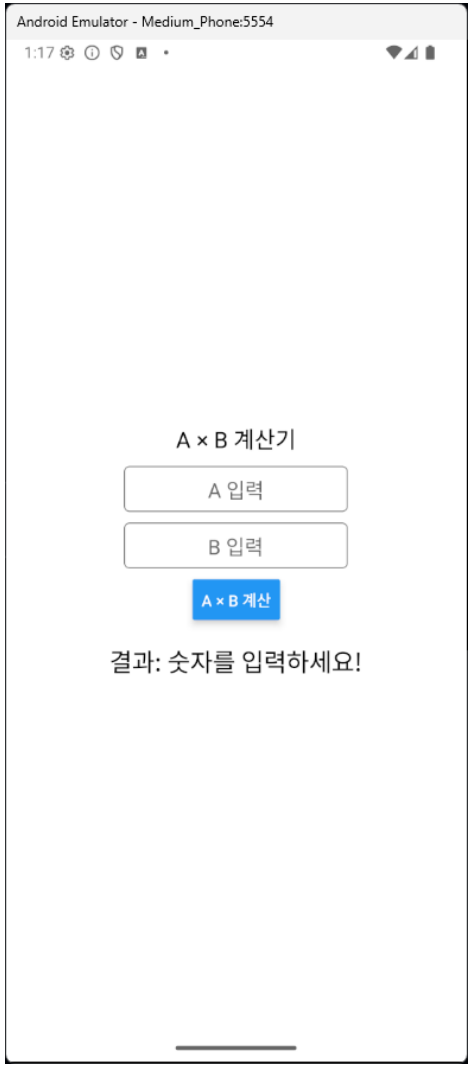

export default function App() {
  const inputRef = useRef(null);


  const focusInput = () => {
    inputRef.current.focus(); // TextInput에 포커스
  };


  return (
    <View style={styles.container}>
      <TextInput style={styles.text} ref={inputRef} placeholder="이름을 입력하세요" />
      <Button title="포커스 이동" onPress={focusInput} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex:1,
    alignItems:'stretch',
    justifyContent:'center',
    padding:30,
  },
 text:{
   fontSize: 30,
 }
});
```

3. useRef: 예제 3




```
import React, { useState, useRef } from "react";
import { StyleSheet, View, Text, TextInput, Button } from "react-native";

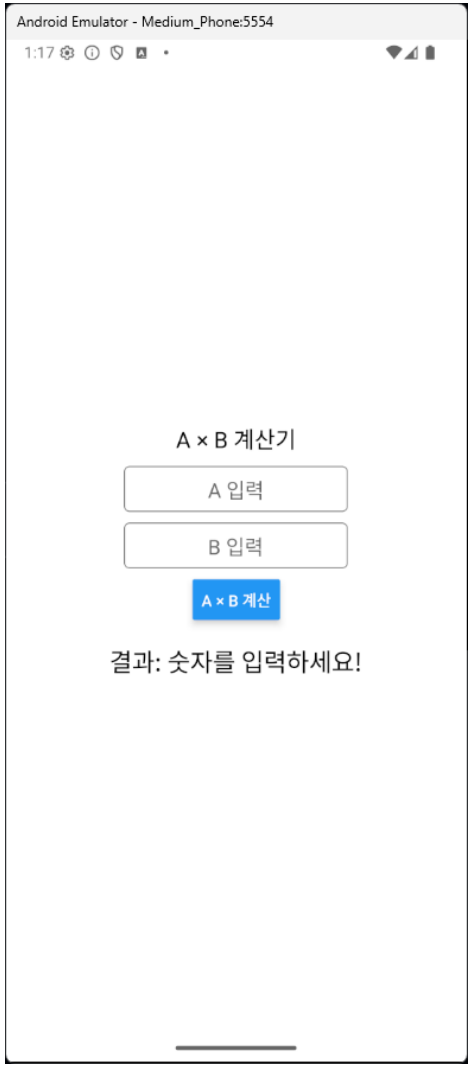
export default function App() {
  const inputRef = useRef(null);
  const focusInput = () => {
    // ref가 존재할 때만 포커스
    inputRef.current?.focus();
  };

  const [a, setA] = useState('');          // 첫 번째 숫자
  const [b, setB] = useState('');          // 두 번째 숫자
  const [result, setResult] = useState(null); // 계산 결과

  const multiply = () => {
    const numA = parseFloat(a);
    const numB = parseFloat(b);


    if (isNaN(numA) || isNaN(numB)) {
      setResult('숫자를 입력하세요!');
    } else {
      setResult(numA * numB);
    }
  };
```

3. useRef: 예제 3




```
const handlePress = () => {
    multiply();       // 계산
    focusInput();     // A 입력창으로 포커스 이동 (선택)
  };
  return (
    <View style={styles.container}>
      <Text style={{ fontSize: 20, marginBottom: 10 }}>A × B 계산기</Text>
      <TextInput
        ref={inputRef}
        style={styles.input}
        keyboardType="numeric"
        placeholder="A 입력"
        value={a}
        onChangeText={setA}
        returnKeyType="next"
        onSubmitEditing={() => { /* 엔터키로 다음 입력으로 이동시키고 싶으면 여기에 */ }}
       />

      <TextInput
        style={styles.input}
        keyboardType="numeric"
        placeholder="B 입력"
        value={b}
        onChangeText={setB} />
      <Button title="A × B 계산" onPress={handlePress} />
      {result !== null && (
        <Text style={{ fontSize: 22, marginTop: 20 }}>결과: {result}</Text>
      )}
    </View>
  );
}
```

4. useEffect

- useEffect 생명주기 제어 컴포넌트가 실행, 변경, 종료 때 코드 실행

  - 예제 코드

```
useEffect( [ ] => {
    console.log( "앱이 실행되었습니다 !(처음 한 번만)" );
}, [ dependency] );
```

어떤 값이 바뀔 때 다시 실행할지 지정

4. useEffect: 예제 1



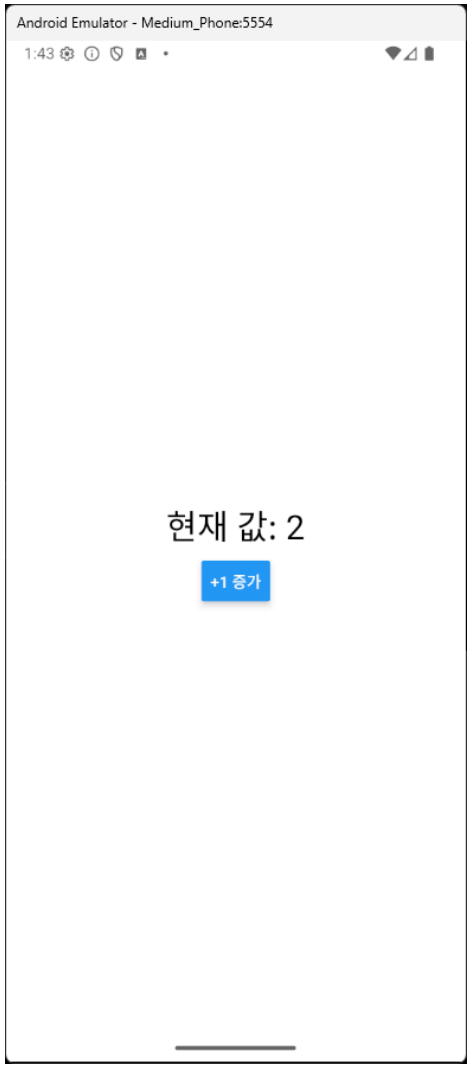

```
import React, { useEffect } from "react";
import { StyleSheet, View, Text } from "react-native";


export default function App() {
  useEffect(() => {
    console.log("앱이 실행되었습니다! (처음 한 번만)");
  }, []); // 빈 배열 → 최초 1회 실행


  return (
    <View style={styles.container}>
      <Text style={styles.text}>useEffect 예제</Text>
    </View>
  );
}


const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',   // 가운데 정렬이 더 보기 좋아요
    justifyContent: 'center',
    padding: 30,
  },
  text:{
    fontSize:30,
  }
});
```

4. useEffect: 예제 2




```
import React, { useState, useEffect } from "react";
import { StyleSheet, View, Text, Button } from "react-native";

export default function App() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log("count 값이 바뀌었습니다:", count);
  }, [count]); // count가 바뀔 때마다 실행됨

  return (
    <View style={styles.container}>
      <Text style={styles.text}>현재 값: {count}</Text>
      <Button title="+1 증가" onPress={() => setCount(count + 1)} />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',    // 가운데 정렬이 더 보기 좋아요
    justifyContent: 'center',
    padding: 30,
  },
  text:{
    fontSize:30,
    paddingBottom:10,
  }
});
```

5. useMemo

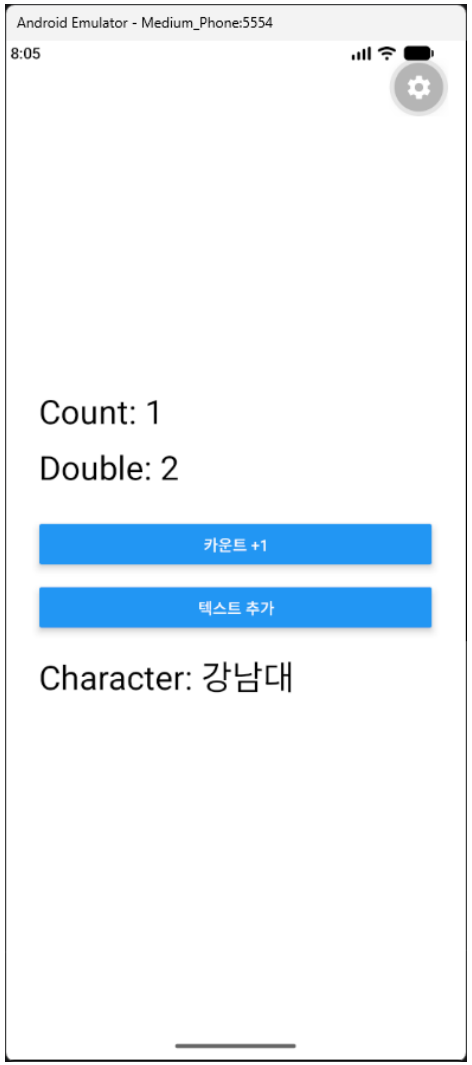
- useMemo 참조관리 불필요한 재 계산 방지(성능 최적화)

  - 예제 코드

```
const memoizedValue = useMemo( ( ) => {
    return 계산식; // 반환값을 기억함
}, [ dependency] );
```

어떤 값이 바뀔 때 다시 계산

5. useMemo:



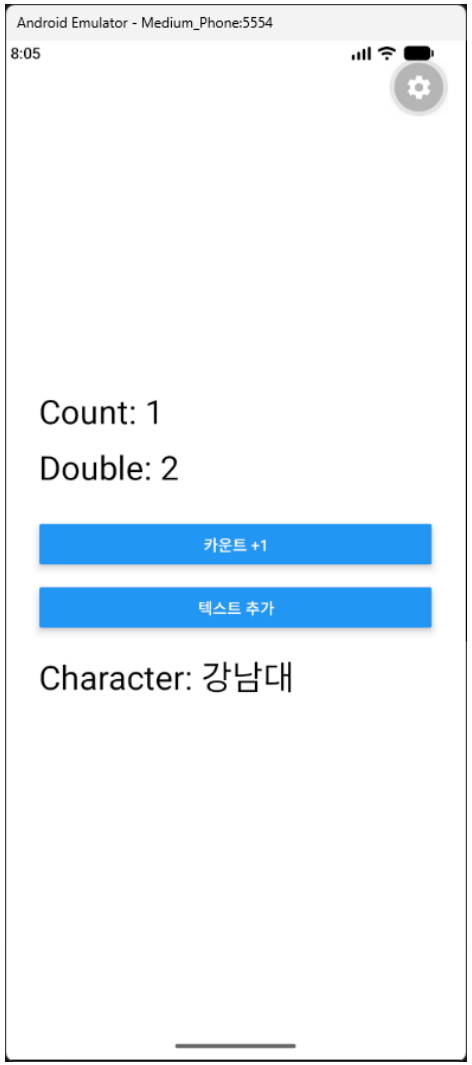

```
import React, { useState, useMemo } from "react";
import { StyleSheet, View, Text, Button } from "react-native";

export default function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  const double = useMemo(() => {
    console.log("double 계산 실행!");
    return count * 2;
  }, [count]);

  return (
    <View style={styles.container}>
      <Text style={styles.text}>Count: {count}</Text>
      <Text style={styles.text}>Double: {double}</Text>
      <Text style={{marginBottom:1}} />
      <Button title="카운트 +1" onPress={() => setCount(count + 1)} />
      <Text style={{marginBottom:1}} />
      <Button title="텍스트 변경" onPress={() => setText(text + '강남대 ')} />
      <Text style={{marginBottom:1}} />
      <Text style={styles.text}>Character: {text}</Text>
    </View>
  );
}
```

5. useMemo:




```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'stretch',
    justifyContent: 'center',
    padding: 30,
  },
  text:{
    fontSize:30,
    paddingBottom:10,
  }
});
```

6. 실습

1. StyleSheet 적용

2. 버튼 누르면 버튼 값 표시

▪ **상세조건**

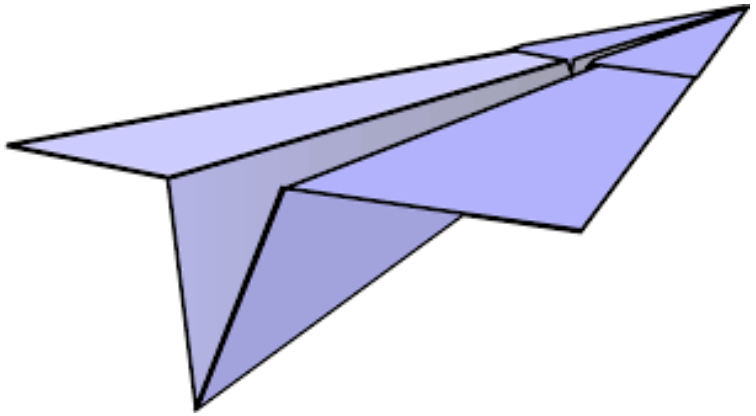
- backgroundColor: 'black'

- `displayText: Color:` 'white'
  FontSize: 84
  textAlign: 'right'

- Button: height:64
  number Button backgroundColor: #2f2f2f
  operatorButton backgroundColor: #F5A623
  utilityButton backgroundColor: #555
  buttonText: color: 'white'
  fontSize: 30
  fontweight: '500'

중간고사 안내

- 일시: 2026년 10월 22일
- 장소: 후 103
- 시간:
- 범위
  1) 강의 자료 2주 ~ 7주.



Q&A

E-mail: jungjongsoo@yahoo.com